Practical 1: BFS and DFS

```
import java.util.*;

public class P1_BFS_DFS {

    static void bfs(Map<Integer, List<Integer>> g, int start) {
        Queue<Integer> q = new LinkedList<>();
        Set<Integer> visited = new HashSet<>();

        q.add(start);
        visited.add(start);

        System.out.print("BFS: ");

        while (!q.isEmpty()) {
            int u = q.poll();
            System.out.print(u + " ");

            for (int v : g.get(u)) {
                if (!visited.contains(v)) {
                    visited.add(v);
                    q.add(v);
                }
            }
        }
        System.out.println();
    }
```

```
static void dfs(Map<Integer, List<Integer>> g,
                int u, Set<Integer> visited) {
    visited.add(u);
    System.out.print(u + " ");

    for (int v : g.get(u)) {
        if (!visited.contains(v)) {
            dfs(g, v, visited);
        }
    }
}

public static void main(String[] args) {

    Map<Integer, List<Integer>> g =
        new LinkedHashMap<>();

    g.put(0, Arrays.asList(1, 2));
    g.put(1, Arrays.asList(0, 3, 4));
    g.put(2, Arrays.asList(0, 5));
    g.put(3, Arrays.asList(1));
    g.put(4, Arrays.asList(1, 5));
    g.put(5, Arrays.asList(2, 4));

    bfs(g, 0);

    System.out.print("DFS: ");
    dfs(g, 0, new HashSet<>());
    System.out.println();
```

```
    }
}
```

Practical 2: A* and Greedy Best-First Search

```
import java.util.*;

public class P2_AStar_Greedy {

    static int[][] grid = {
        {0,0,0,0,0},
        {0,1,1,0,0},
        {0,0,0,0,0},
        {0,1,0,1,0},
        {0,0,0,0,0}
    };

    static int[][] dirs = {
        {1,0},{-1,0},{0,1},{0,-1}
    };

    static int h(int r, int c, int gr, int gc) {
        return Math.abs(r-gr) + Math.abs(c-gc);
    }

    static class Node {
        int r,c,g,f;

        Node(int r,int c,int g,int f) {
            this.r=r;
```

```
        this.c=c;
        this.g=g;
        this.f=f;
    }
}

static void search(boolean astar) {

    int n=grid.length;
    int sr=0, sc=0, gr=4, gc=4;

    int[][] dist=new int[n][n];

    for(int[] row:dist)
        Arrays.fill(row,Integer.MAX_VALUE);

    PriorityQueue<Node> pq =
        new PriorityQueue<>((a,b) ->
            Integer.compare(a.f,b.f));

    dist[sr][sc]=0;

    pq.add(new Node(sr,sc,0,
        h(sr,sc,gr,gc)));

    boolean[][] done=new boolean[n][n];

    while(!pq.isEmpty()) {
```

```
Node cur=pq.poll();

if(done[cur.r][cur.c])
    continue;

done[cur.r][cur.c]=true;

if(cur.r==gr && cur.c==gc) {
    System.out.println(
        (astar ? "A*" : "Greedy Best-First")
        + " reached goal. Path cost = "
        + cur.g);
    return;
}

for(int[] d:dirs) {

    int nr=cur.r+d[0];
    int nc=cur.c+d[1];

    if(nr>=0 && nr<n &&
       nc>=0 && nc<n &&
       grid[nr][nc]==0) {

        int ng=cur.g+1;

        if(ng<dist[nr][nc]) {

            dist[nr][nc]=ng;
```

```
                    int priority=astar
                        ? ng+h(nr,nc,gr,gc)
                        : h(nr,nc,gr,gc);

                    pq.add(new Node(
                        nr,nc,ng,priority));
                }
            }
        }
    }

    System.out.println("Goal not reachable.");
  }

  public static void main(String[] args) {

    System.out.println(
        "0 = free cell, 1 = obstacle");

    search(true);
    search(false);
  }
}
```

Practical 3: Hill Climbing and Simulated Annealing

```
import java.util.*;

public class P3_HillClimbing_SimulatedAnnealing {
```

```
static int conflicts(int[] q) {
    int c=0;

    for(int i=0;i<q.length;i++) {
        for(int j=i+1;j<q.length;j++) {

            if(q[i]==q[j] ||
               Math.abs(q[i]-q[j])==Math.abs(i-j)) {
                c++;
            }
        }
    }
    return c;
}

static int[] randomBoard(Random rnd,int n) {
    int[] q=new int[n];

    for(int i=0;i<n;i++)
        q[i]=rnd.nextInt(n);

    return q;
}

static int[] hillClimb() {

    Random rnd=new Random(7);
    int n=8;
```

```
int[] q=randomBoard(rnd,n);

for(int step=0;step<10000;step++) {

    int best=conflicts(q);
    int bi=-1,bv=-1;

    for(int col=0;col<n;col++) {

        int old=q[col];

        for(int row=0;row<n;row++) {

            q[col]=row;
            int score=conflicts(q);

            if(score<best) {
                best=score;
                bi=col;
                bv=row;
            }
        }

        q[col]=old;
    }

    if(bi==-1)
        break;
```

```
            q[bi]=bv;

            if(conflicts(q)==0)
                break;
        }

        return q;
    }

    static int[] simulatedAnnealing() {

        Random rnd=new Random(11);
        int n=8;
        int[] q=randomBoard(rnd,n);

        double temp=10.0;

        for(int step=0;step<100000 &&
            conflicts(q)>0;step++) {

            int col=rnd.nextInt(n);
            int old=q[col];
            int oldScore=conflicts(q);

            int newRow=rnd.nextInt(n);
            q[col]=newRow;

            int newScore=conflicts(q);
```

```
        if(newScore>oldScore &&
           rnd.nextDouble() >
           Math.exp((oldScore-newScore)/temp)) {

            q[col]=old;
        }

        temp*=0.9995;

        if(temp<0.001)
            temp=0.001;
    }

    return q;
}

static void show(String name,int[] q) {

    System.out.println(
        name+" board: "+Arrays.toString(q));

    System.out.println(
        "Number of conflicts: "+conflicts(q));
}

public static void main(String[] args) {

    show("Hill Climbing",hillClimb());
```

```
        show("Simulated Annealing",
             simulatedAnnealing());

        System.out.println(
            "0 conflicts means a valid 8-Queens solution.");
    }
}
```

Practical 4: Tic-Tac-Toe using Minimax and Alpha-Beta Pruning

```
public class P4_TicTacToe_Minimax {

    static char[] b = {
        'X','O','X',
        'O','X',' ',
        ' ',' ',' '
    };

    static char winner() {

        int[][] lines = {
            {0,1,2},{3,4,5},{6,7,8},
            {0,3,6},{1,4,7},{2,5,8},
            {0,4,8},{2,4,6}
        };

        for(int[] x:lines) {

            if(b[x[0]]!=' ' &&
               b[x[0]]==b[x[1]] &&
```

```
            b[x[1]]==b[x[2]]) {

                return b[x[0]];
            }
        }

        return ' ';
    }

    static int minimax(
        boolean maximizing,int alpha,int beta) {

        char w=winner();

        if(w=='X') return 10;
        if(w=='O') return -10;

        boolean full=true;

        for(char c:b)
            if(c==' ') full=false;

        if(full) return 0;

        if(maximizing) {

            int best=-1000;

            for(int i=0;i<9;i++) {
```

```
        if(b[i]==' ') {

            b[i]='X';

            best=Math.max(best,
                minimax(false,alpha,beta));

            b[i]=' ';

            alpha=Math.max(alpha,best);

            if(beta<=alpha)
                break;
        }
    }

    return best;

} else {

    int best=1000;

    for(int i=0;i<9;i++) {

        if(b[i]==' ') {

            b[i]='O';
```

```
                best=Math.min(best,
                    minimax(true,alpha,beta));

                b[i]=' ';

                beta=Math.min(beta,best);

                if(beta<=alpha)
                    break;
            }
        }

        return best;
    }
}

public static void main(String[] args) {

    int best=-1000;
    int move=-1;

    for(int i=0;i<9;i++) {

        if(b[i]==' ') {

            b[i]='X';

            int score=minimax(
                false,-1000,1000);
```

```
                b[i]=' ';

                if(score>best) {
                    best=score;
                    move=i;
                }
            }
        }

        System.out.println("Board positions:");

        for(int i=0;i<9;i++) {

            String cell = b[i]==' '
                ? Integer.toString(i+1)
                : Character.toString(b[i]);

            System.out.print(
                cell+(i%3==2 ? "\n" : " | "));
        }

        System.out.println(
            "AI (X) chooses position: "+(move+1));

        System.out.println(
            "Minimax score: "+best);
    }
}
```

Practical 5: Constraint Satisfaction Problem (Map Coloring)

```
import java.util.*;

public class P5_MapColoring_CSP {

    static String[] regions = {
        "A","B","C","D"
    };

    static String[] colors = {
        "Red","Green","Blue"
    };

    static Map<String,List<String>> adj =
        new LinkedHashMap<>();

    static Map<String,String> assigned =
        new LinkedHashMap<>();

    static boolean safe(String region,String color) {

        for(String neighbor:adj.get(region)) {

            if(color.equals(assigned.get(neighbor)))
                return false;
        }

        return true;
```

```
}

static boolean solve(int index) {

    if(index==regions.length)
        return true;

    String region=regions[index];

    for(String color:colors) {

        if(safe(region,color)) {

            assigned.put(region,color);

            if(solve(index+1))
                return true;

            assigned.remove(region);
        }
    }

    return false;
}

public static void main(String[] args) {

    adj.put("A",Arrays.asList("B","C"));
    adj.put("B",Arrays.asList("A","C","D"));
```

```
        adj.put("C",Arrays.asList("A","B","D"));
        adj.put("D",Arrays.asList("B","C"));

        if(solve(0)) {

            System.out.println(
                "Valid map coloring: "+assigned);

        } else {

            System.out.println("No solution found.");
        }
    }
}
```

Practical 6: Wumpus World using Propositional Logic and Forward Chaining

```
import java.util.*;

public class P6_Wumpus_ForwardChaining {

    public static void main(String[] args) {

        Set<String> facts = new LinkedHashSet<>();

        // Initial facts
        facts.add("Breeze(1,1)");
        facts.add("Stench(2,1)");
        facts.add("NoPit(1,1)");
        facts.add("NoWumpus(1,1)");
```

```
        System.out.println("Initial facts: "+facts);

        // New observations
        facts.add("NoBreeze(1,2)");
        facts.add("NoStench(1,2)");

        // Forward chaining rules
        if(facts.contains("NoBreeze(1,2)")) {

            facts.add("NoPit(1,1)");
            facts.add("NoPit(1,3)");
            facts.add("NoPit(2,2)");
        }

        if(facts.contains("NoStench(1,2)")) {

            facts.add("NoWumpus(1,1)");
            facts.add("NoWumpus(1,3)");
            facts.add("NoWumpus(2,2)");
        }

        System.out.println(
            "Facts after forward chaining: "+facts);

        System.out.println(
            "Inference: Cells inferred safe from pit/Wumpus clues are marked
NoPit/NoWumpus.");
    }
```

```
}
```

Practical 7: First-Order Logic, Unification and Chaining

```
import java.util.*;

public class P7_FOL_Unification_Chaining {

    static Map<String,String> unify(
        String pattern,String fact) {

        String pArgs = pattern.substring(
            pattern.indexOf('(')+1,
            pattern.lastIndexOf(')'));

        String fArgs = fact.substring(
            fact.indexOf('(')+1,
            fact.lastIndexOf(')'));

        String[] p = pArgs.split("\\s*,\\s*");
        String[] f = fArgs.split("\\s*,\\s*");

        if(p.length!=f.length)
            return null;

        Map<String,String> theta =
            new LinkedHashMap<>();

        for(int i=0;i<p.length;i++) {
```

```
        if(p[i].startsWith("?")) {
            theta.put(p[i],f[i]);
        }

        else if(!p[i].equals(f[i])) {
            return null;
        }
    }

    return theta;
}

public static void main(String[] args) {

    String rule =
        "Parent(?x, ?y) AND Parent(?y, ?z) => Grandparent(?x, ?z)";

    String fact1 = "Parent(Alice, Bob)";
    String fact2 = "Parent(Bob, Carol)";

    System.out.println("Rule: "+rule);

    System.out.println(
        "Unification 1: "+
        unify("Parent(?x,?y)",fact1));

    System.out.println(
        "Unification 2: "+
        unify("Parent(?y,?z)",fact2));
```

```
        System.out.println(
            "Forward chaining: Alice is Grandparent of Carol.");

        System.out.println(
            "Backward chaining: To prove Grandparent(Alice,Carol), check
Parent(Alice,Bob) and Parent(Bob,Carol).");
    }
}
```

Practical 8: Classical Planning using STRIPS

```
import java.util.*;

public class P8_STRIPS_GridPlanning {

    static class State {

        int x,y;

        State(int x,int y) {
            this.x=x;
            this.y=y;
        }

        public boolean equals(Object o) {

            if(!(o instanceof State))
                return false;
```

```
        State s=(State)o;

        return x==s.x && y==s.y;
    }

    public int hashCode() {
        return Objects.hash(x,y);
    }

    public String toString() {
        return "("+x+","+y+")";
    }
}

public static void main(String[] args) {

    State start=new State(0,0);
    State goal=new State(2,2);

    int[][] moves = {
        {1,0},{0,1},{-1,0},{0,-1}
    };

    String[] names = {
        "RIGHT","DOWN","LEFT","UP"
    };

    Queue<State> q=new LinkedList<>();
```

```
Map<State,State> parent=new HashMap<>();

Map<State,String> action=new HashMap<>();

q.add(start);
parent.put(start,null);

while(!q.isEmpty()) {

    State s=q.poll();

    if(s.equals(goal))
        break;

    for(int i=0;i<4;i++) {

        State t=new State(
            s.x+moves[i][0],
            s.y+moves[i][1]);

        if(t.x>=0 && t.x<=2 &&
           t.y>=0 && t.y<=2 &&
           !parent.containsKey(t)) {

            parent.put(t,s);
            action.put(t,names[i]);

            q.add(t);
        }
```

```
            }
        }

        List<String> plan=new ArrayList<>();

        State cur=goal;

        while(!cur.equals(start)) {

            plan.add(action.get(cur));
            cur=parent.get(cur);
        }

        Collections.reverse(plan);

        System.out.println(
            "Initial state: "+start);

        System.out.println(
            "Goal state: "+goal);

        System.out.println(
            "STRIPS-style plan: "+plan);

        System.out.println(
            "Actions have preconditions (valid adjacent cell) and effects (new robot
position).");
    }
}
```

Practical 9: Bayesian Network using Bayes' Rule

```
public class P9_BayesianNetwork {

    public static void main(String[] args) {

        // Probability of rain
        double pRain = 0.2;

        // Probability of wet grass if raining
        double pWetGivenRain = 0.9;

        // Probability of wet grass without rain
        double pWetGivenNoRain = 0.1;

        // Total probability of wet grass
        double pWet =
            pWetGivenRain * pRain +
            pWetGivenNoRain * (1-pRain);

        // Bayes' Rule
        double pRainGivenWet =
            (pWetGivenRain * pRain) / pWet;

        System.out.printf(
            "P(Rain) = %.2f%n",pRain);

        System.out.printf(
            "P(WetGrass) = %.2f%n",pWet);
```

```
        System.out.printf(
            "P(Rain | WetGrass) = %.4f (%.2f%%)%n",
            pRainGivenWet,pRainGivenWet*100);

        System.out.println(
            "Inference: If grass is wet, probability of rain is updated using Bayes' Rule.");
    }
}
```

Practical 10: Decision Tree Classification

```
import java.util.*;

public class P10_DecisionTree_Java {

    static class Record {

        String outlook;
        String result;

        Record(String o,String r) {
            outlook=o;
            result=r;
        }
    }

    static String predict(String outlook) {

        if(outlook.equalsIgnoreCase("Sunny"))
```

```
        return "No";

    if(outlook.equalsIgnoreCase("Rainy"))
        return "Yes";

    if(outlook.equalsIgnoreCase("Overcast"))
        return "Yes";

    return "Unknown";
}

public static void main(String[] args) {

    List<Record> data = Arrays.asList(

        new Record("Sunny","No"),
        new Record("Rainy","Yes"),
        new Record("Overcast","Yes"),
        new Record("Sunny","No"),
        new Record("Rainy","Yes")
    );

    System.out.println(
        "Sample training dataset (Outlook -> Play):");

    for(Record r:data) {

        System.out.println(
            r.outlook+" -> "+r.result);
```

```
        }

        String test="Sunny";

        System.out.println(
            "Test input: Outlook = "+test);

        System.out.println(
            "Decision Tree prediction: Play = "+
            predict(test));
    }
}
```